Vantora AI — Technical Documentation

Deliverable 01 · FinTechathon Topic E — SME Finance Copilot ("AI as a Financial Participant")

This document specifies the system architecture, the stochastic cash-flow model, the financing eligibility screen, the Mixed-Integer Linear Program that produces the recovery plan, and the identity, approval and sandbox-execution layer that governs what happens next — at the level of detail required to re-derive every published figure from the source.

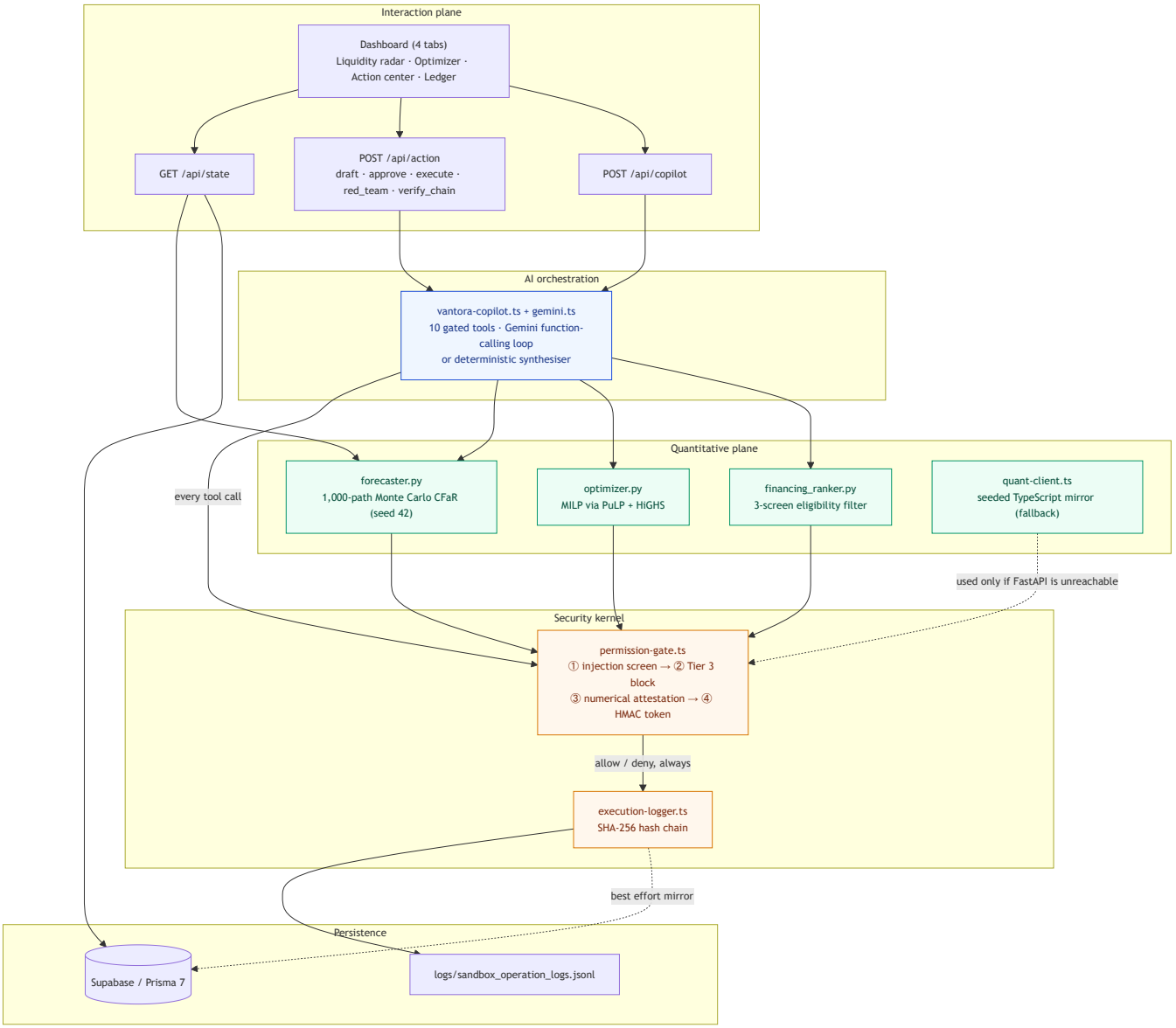
> **Data provenance.** The ledger is synthetic: Smart Jeans Manufacturing Ltd. is invented, and real company names (Zara, Kenneth Cole, Atradius, Coface, …) are used illustratively with no affiliation implied. The canonical day-14 scenario was hand-designed to illustrate a payroll collision. Its behaviour beyond that one shape is exercised across three regimes and all 216 preset × stress-dial combinations by the test suite and the solver benchmark (§4.5); no real company's books have been run through it.

1. Architectural principle

The model never computes and never executes.

Two failure modes make a conversational agent unusable in corporate treasury. A model that states `$45,000` where the solver said `$42,355` has produced a plausible, auditable-looking, wrong instruction. A model holding payment tools with an attacker-controlled invoice memo in its context is one injection away from wiring funds. Behavioural mitigation (better prompts, more alignment) reduces the rate of both; it does not bound them.

Vantora's mitigation is structural. Every figure originates in a deterministic solver and is re-verified against that solver before it reaches a human. Every side effect passes through a permission gate that fails closed. The model's authority is reduced to *explanation*.



1.1 Plane responsibilities

Plane	Module	Responsibility	May it originate a number?
Quantitative	`quant_engine/` (FastAPI, NumPy/SciPy, PuLP + HiGHS)	Monte Carlo CFaR, financing screen, MILP optimisation	**Yes — sole source**
Quantitative (fallback)	`src/lib/quant-client.ts`	Seeded TypeScript mirror when FastAPI is unreachable	Yes, and the UI declares when it is active
Security	`src/lib/security/`, `src/lib/sandbox/`	Tier classification, HMAC HITL tokens, numerical attestation, injection screen, hash-chained ledger	No — it only *verifies*
AI	`src/lib/agent/vantora-copilot.ts`	Explanation, drafting, tool orchestration	**No**
Interaction	`src/app/`, `src/components/vantora/`	Rendering, human approval capture	No

1.2 Wire contract

The HTTP contract is snake_case throughout because it mirrors the Pydantic models in `quant_engine/schemas.py` field for field. There is no mapping layer, so there is nothing to drift. All Pydantic models declare `extra="forbid"`: an unexpected field is a 422, not a silently ignored typo.

```
POST /api/quant/forecast   SmeFinancialStateInput -> ForecastResultOutput
POST /api/quant/optimize   SmeFinancialStateInput -> OptimizationResultOutput
GET  /health
```

The TypeScript bridge imposes a 2.5-second timeout and `cache: "no-store"`. Any failure — offline, timeout, non-JSON — returns `null` and the caller falls through to the mirror. The two engines are asserted to agree on the crunch day and the required cash in `src/lib/__tests__/quant-client.test.ts`.

---

2. Stochastic cash-flow model (`forecaster.py`)

2.1 Simulation structure

Let $H = 30$ be the horizon in days and $N = 1000$ the number of trajectories. The generator is `np.random.default_rng(seed=42)`, so every run is bit-reproducible — asserted by a test that runs the forecast twice and compares percentile arrays element-wise.

For each simulation $s \in \{1..N\}$ and day $d \in \{1..H\}$:

$$B(s, d) = C_0 + \Sigma_{i \in AR}\ A_i \cdot 1[\ T_i(s) \le d\ ] \ -\ \Sigma_{j \in AP}\ P_j \cdot 1[\ d_j \le d\ ]$$

Symbol	Meaning
C_0	opening cash (`current_cash`)
A_i , P_j	invoice / payable amounts
$T_i(s)$	**stochastic** arrival day of receivable `i` in trajectory `s`
d_j	**deterministic** due day of payable `j`

Only receivables are stochastic. Payables are contractual obligations with known dates — modelling them as random would understate the risk by smearing a hard collision across several days.

Inflows are accumulated with `np.add.at` over an `(N, H+2)` array, so the whole simulation is vectorised rather than looped per trajectory.

2.2 B2B payment-delay distribution

The arrival day of receivable `i` is its due day plus a delay drawn from a reliability-stretched truncated normal, plus an occasional heavy dispute tail:

```
stretch_i  = 2.0 − r_i                          r_i = reliability_score ∈ (0, 1]
μ_i        = mean_i  · stretch_i
σ_i        = max(std_i · stretch_i, 1e-6)

D_i(s) ~ TruncNormal(μ_i, σ_i;  lower = 0, upper = ∞)          # scipy.stats.truncnorm, a = −μ_i/σ_i
U_i(s) ~ Uniform(0, 1)
L_i(s) ~ LogNormal(ln 7, 0.5)                                   # dispute tail, median 7 days

T_i(s) = ⌈ d_i + D_i(s) + 1[ U_i(s) < 1 − r_i ] · L_i(s) ⌉
```

Systemic delay correlation (`correlation_rho`)

Independent invoice delays understate SME risk in the one scenario that actually kills a firm: a sector-wide slowdown in which the *whole* receivables book slips at once. The model therefore admits a single common factor, off by default:

```
Z_sys(s)  ~ Normal(0, 1)                                one draw per trajectory, shared by every invoice
Z_id(s,i) ~ Normal(0, 1)                                idiosyncratic, per invoice
Z(s,i)    = √ρ · Z_sys(s) + √(1−ρ) · Z_id(s,i)          ρ ∈ [0, 0.95], default 0.0
D_i(s)    = TruncNormal_ppf( Φ(Z(s,i)); μ_i, σ_i, lower = 0 )
```

`Z(s,i)` has unit variance for every `ρ`, so raising `ρ` redistributes dispersion from idiosyncratic to systemic without changing each invoice's marginal delay distribution. This is a Gaussian copula: the marginals are preserved exactly, only the dependence structure moves.

`ρ = 0` takes a different code path on purpose. The copula reaches the same marginal through `truncnorm.ppf(norm.cdf(z))`, but not through the same RNG draws, and every canonical figure is pinned to the independent sampler. At `ρ = 0` the engine calls `truncnorm.rvs` exactly as it always did, so the default output is byte-identical — asserted by `test_rho_zero_leaves_the_canonical_trajectories_untouched`.

Measured effect on the pytest engine fixture (mean P90–P10 spread across the 30-day window):

`ρ`	Mean P90–P10 spread	Reading
0.0	$43,233	invoices slip independently; the book diversifies itself
0.3	$44,967	moderate sector linkage
0.6	$48,867	sector stress — late receivables arrive late *together*

The dashboard exposes this as a live control on Tab 1; `GET /api/state?rho=0.6` re-runs the forecast through the same Python path.

Three further modelling decisions are worth stating explicitly:

1. **Reliability stretches both location and scale.** A counterparty with `r = 0.72` is not merely later on average (`stretch = 1.28`); it is *less predictable*. Stretching only the mean would model a reliably-late payer, which is a materially easier risk to finance.
2. **The distribution is truncated, not clipped.** `truncnorm` re-samples the forbidden region rather than piling probability mass at zero, so "pays exactly on time" does not get a spurious spike.
3. **The dispute tail is log-normal and fires with probability `1 − r_i`.** Real B2B receivables have a small population of invoices that go into query and resolve weeks late. A single normal cannot produce that shape; a

symmetric model underestimates the P10 tail, which is precisely the quantity the whole system exists to measure.

2.3 Percentile trajectories and derived metrics

For each day `d`, the P10/P50/P90 trajectories are taken across the `N` simulations using NumPy's default linear-interpolation percentile:

```
P₁₀(d) = percentile( { B(s,d) : s = 1..N }, 10 )      (likewise P₅₀, P₉₀)
```

Output	Definition
`first_shortfall_day`	`min { d : P₁₀(d) < buffer }`, else `null`
`p10_shortfall_amount`	`max(0, buffer − min_d P₁₀(d))` — the deepest drawdown, not merely the first breach
`shortfall_probability`	`(1/N) · #{ s : min_d B(s,d) < buffer }`
`invoice_p90_payment_day`	`percentile({Tᵢ(s)}, 90)` — the pessimistic arrival used by the optimiser
`decision_day`	`max(1, first_shortfall_day − 3)` — see below
`p10_ci_95`	distribution-free 95% CI on the crunch-day P10 — see below
`days_sales_outstanding`	`total_AR / (monthly_revenue / 30)`
`days_payable_outstanding`	`total_AP / daily_COGS` — see below
`runway_days_p50`	first day with `P₅₀(d) < 0`, else `H`

P10 is the planning line. A plan built on the median is a plan that fails half the time; `restore_p10_to_safe_buffer` is the MILP's binding constraint for exactly this reason.

DPO is payables over COGS, not over revenue

Days payable outstanding measures how long the firm takes to pay for *what it buys*. Dividing accounts payable by daily revenue divides a cost-basis numerator by a price-basis denominator, understating DPO by the entire gross margin and making the cash-conversion gap look structurally worse than it is:

```
daily_revenue = monthly_revenue / 30
daily_COGS    = monthly_revenue · (1 − gross_margin) / 30        gross_margin default 0.35

DSO = total_AR / daily_revenue
DPO = total_AP / daily_COGS
cash_conversion_gap_days = DSO − DPO
```

On the canonical dataset this moves DPO from 29.63 to **35.27 days** and the cash-conversion gap from `+2.48` to **`−3.16` days**. The corrected figure strengthens the diagnosis rather than weakening it: a firm that pays suppliers *slower* than it collects has no structural working-capital deficit, which is precisely why day 14 is a timing collision and not a trend. `gross_margin` defaults to 0.35 so every pre-existing caller is unaffected; the Smart Jeans dataset declares **0.16**, because in cut-make-trim apparel direct labour sits *inside* COGS. At 16% of $185,000 the plant clears roughly $29,600 of gross profit a month against $9,500 of rent and $4,200 of utilities — which is why a $74,700 timing gap is existential here rather than inconvenient.

Decision day — the lever deadline, not the failure date

`first_shortfall_day` is when the firm runs out of money. It is the wrong date to act on, because by then every lever that needs counterparty agreement has closed. The payroll file locks 72 hours before the run:

```
decision_day = max(1, first_shortfall_day - 3)          null when no shortfall is detected
```

On the canonical dataset this is **day 11**, not day 14, and it coincides with the declared `payroll_lock_day` (§6.3). §7 prices what the three intervening days cost.

Confidence interval on the P10 estimate

P10 is itself a sample statistic from 1,000 trajectories and carries sampling error. Rather than bootstrap, the interval is read directly off the order statistics — the rank of the `q`-quantile is `Binomial(N, q)`, and its normal approximation brackets it:

```
half_width = 1.96 · √(N · q · (1 − q))                  N = 1000, q = 0.10  →  ±18.59
p10_ci_95  = [ x₍⌊Nq − half_width⌋₎ , x₍⌈Nq + half_width⌉₎ ]     x sorted ascending
```

This is exactly reproducible (no second RNG), costs one sort, and is identical arithmetic in `forecaster.py` and `quant-client.ts`.

On the canonical dataset the interval is `[-$49,700, -$49,700]` — zero width. That is a correct result, not a defect: at least 80% of the runs end day 14 on the same balance, so the 10th percentile has no sampling error to report at this N. It does **not** mean no receivable can arrive. Some runs do differ (§2.4), but too few to move a percentile.

2.4 Canonical result

Smart Jeans Manufacturing Ltd. — C_0 = $52,000, buffer = $25,000, credit score 672, revenue $185,000/month ($2.22M annual), gross margin 16%.

```
Day 13:  P₁₀ = $53,300                                  above buffer
Day 14:  P₁₀ = P₅₀ = P₉₀ = −$49,700                     $68,000 payroll + $35,000 Vulcan tooling
         required = buffer − P₁₀(14) = $25,000 − (−$49,700) = $74,700
         shortfall_probability = 1.000
```

Kenneth Cole ($52,000, due d9) has a P90 arrival of day 38, and Zara ($43,000, due d11) of day 32. In the seeded 1,000-run simulation, 947 runs end day 14 at exactly −$49,700. 37 runs (3.7%) see Zara or Kenneth Cole pay early enough, which lifts the balance to −$6,700 or +$2,300. 16 runs (1.6%) see a small early invoice slip, which takes it down to −$75,700 or −$83,700. No run reaches the $25,000 buffer, so the probability is 100.0% *within the model*. Because each tail is under 10% of runs, P10, P50 and P90 all read −$49,700. The uncertainty is real, but it sits on days 3–8 (the small invoices) and days 17–30 (the large ones; the P90–P10 band peaks at $126,000 on day 26). The delay distributions are authored assumptions, and payables and the absence of new sales are fixed. This is certainty inside the model, not a guarantee about real buyers. The dashboard computes the open-band windows from the trajectories (`uncertaintyWindows`) rather than hard-coding this paragraph.

> **A second, more stochastic fixture is kept deliberately.** `quant_engine/tests/conftest.py` exercises a different dataset (C_0 = $48,000, gap $58,000) whose shortfall probability is **98.2%** — a case where a minority of trajectories survive. Validating the forecaster against only the degenerate 100% case would hide a broken percentile implementation.

3. Financing ranker (`financing_ranker.py`)

Screening is deterministic, ordered, and short-circuiting. No model judgement participates; each rejection records the exact predicate that failed.

3.1 The three screens

```
Screen 1 — Creditworthiness
    credit_score < min_credit_score_f          -> reject
    reason: "credit score {s} below minimum {m}"

Screen 2 — Funding speed against the deadline
    funding_days_f ≥ crunch_day                -> reject
    reason: "funds in {k}d, needed by day {c}"

Screen 3 — Collateral coverage (collateralised products only)
    pledgeable_AR = Σ Aᵢ for invoices without QTY_VARIANCE / PRICE_VARIANCE
    available_f = min( max_amount_f , coverage_f · pledgeable_AR )
    available_f ≤ 0                            -> reject
```

Screen 2 is strict (`≥`, not `>`): capital that lands *on* the crunch day has not cleared in time to meet an obligation falling due that day.

3.2 Cost model

The bridge term is `T = 30` days. For a draw of `a` dollars on facility `f`:

```
cost_f(a)        = origination_fee_f · a  +  a · APR_f · (T / 365)
cost_per_dollar_f = origination_fee_f     +      APR_f · (T / 365)
effective_APR_f  = ( cost_f(a) / a ) · (365 / T)
```

`cost_per_dollar_f` is the constant the MILP uses as the linear coefficient on `z_f`. Note that `effective_APR` is the honest comparator: a 4% origination fee on a 30-day advance is not a 4% cost, it is a ~49 percentage-point annualised addition. This is why QuickFloat's quoted 46% APR surfaces as **94.7% effective**.

3.3 Canonical screen result

Rank	Facility	Product	Quoted APR	Fee	Funding	Effective APR (30d)	Available	Outcome
1	Meridian Capital Partners	confidential AR line	14.0%	2.0%	2 d	**38.3%**	$80,000	✅ eligible — **$2,345 drawn**
2	QuickFloat Emergency Advance	merchant advance	46.0%	4.0%	2 d	**94.7%**	$90,000	✅ eligible (naive counterfactual only)
3	Heritage SBA Microloan	bridge loan	8.0%	1.0%	21 d	20.2%	$0	❌ Screen 2 — *funds in 21d, needed by day 14*
4	Granite Commercial Factoring	bridge loan	11.0%	1.5%	7 d	29.3%	$0	❌ Screen 1 — *credit score 672 below minimum 700*

Heritage SBA is the cheapest capital on the board and is still refused. **Timing is a hard constraint, not a preference** — a correct screen must be willing to reject the best price.

---

4. MILP recovery optimizer (`optimizer.py`)

4.1 Decision variables

Variable	Domain	Meaning
x_i	$\{0,1\}$	offer an early-payment discount on eligible receivable i
y_j	$\{0,1\}$	defer flexible payable j past the crunch day
z_f	$[0, \text{available}_f] \subset \mathbb{R}$	dollars drawn on eligible facility f

Variables are only created for levers that can actually move the crunch day — the program is built after a feasibility pre-filter:

- x_i exists only if the invoice is discount-eligible, $\text{settles_in_days} \le \text{crunch_day}$, **and** its P90 arrival is *after* the crunch day. The last condition matters: discounting an invoice the pessimistic scenario already has in hand buys nothing and would otherwise show up as free cash.
- y_j exists only if the payable is flexible, $d_j \le \text{crunch_day}$, and the deferral window actually clears the crunch ($d_j + \text{max_deferral_days} > \text{crunch_day}$).
- z_f exists only for facilities that cleared all three screens.

4.2 Program

$$\text{minimise} \quad \sum_i (A_i \cdot \delta_i)\, x_i \;+\; \sum_j (A_j \cdot \phi_j + \rho_j \cdot \Pi)\, y_j \;+\; \sum_f \text{cost_per_dollar}_f \cdot z_f$$

$$\text{subject to} \quad \sum_i A_i(1-\delta_i)\, x_i \;+\; \sum_j A_j\, y_j \;+\; \sum_f z_f \;\ge\; \text{required} \quad \text{(restore_p10_to_safe_buffer)}$$

$$\sum_j \rho_j\, y_j \;\le\; \rho_{\max} \quad \text{(relationship_risk_budget)}$$

$$x_i,\ y_j \in \{0,1\}, \qquad 0 \le z_f \le \text{available}_f$$

Symbol	Meaning	Canonical value
δ_i	early-payment discount rate on invoice i	1.5% – 3.0%
ϕ_j	deferral fee rate on payable j	0.5% – 1.0%
ρ_j	relationship-risk weight of deferring j	0.10 – 0.25
Π	price of one unit of relationship risk (`relationship_risk_price`)	$2,000
$\rho_{\max}$	the SME's relationship-risk budget	0.40
`required`	$\text{buffer} - P_{10}(\text{crunch_day})$	$74,700

Two features deserve comment:

- **Discounting costs the discount but raises the net.** A discounted invoice contributes $A_i(1-\delta_i)$ to the constraint while costing $A_i \cdot \delta_i$ in the objective. Treating the full face value as the gain would overstate recovery by exactly the discount.

- **Goodwill is priced, not merely capped.** $\rho_j \cdot \Pi$ puts relationship damage into the objective in dollars, while ρ_{max} bounds it as a hard constraint. A pure cap would make the solver treat supplier goodwill as free right up to the limit.

Solver. PuLP 4 ships no bundled CBC, so the program is solved through **HiGHS** via the `highspy` wheel: `problem.solve(pulp.getSolver("HiGHS", msg=False))`, status read from `stats.status.name`. The test suite asserts `LpStatusOptimal`.

> **Known approximation.** Each lever is scored against the P10 trajectory rather than re-simulating the Monte Carlo per candidate plan, which is what keeps the program linear. Plan-interaction effects are therefore not modelled. Upgrade path if they begin to matter: re-run the forecaster on the chosen plan and iterate (cutting-plane / sample-average approximation).

4.3 Counterfactual baselines

The optimiser does not merely recommend; it prices the two alternatives so the recommendation arrives with its own proof.

Naive borrowing — draw the entire `required` from the fastest eligible facility, ignoring free working-capital levers.

Do nothing — no financing, so obligations go unfunded. The money runs out on what is due *on and after* the crunch day, payroll first (anything due earlier was still affordable):

```
unfunded = required
for j in sorted({ j : d_j ≥ crunch_day }, by d_j):
    unpaid_j  = min(A_j, unfunded);   unfunded -= unpaid_j
    penalty_j = late_fee_j · unpaid_j  +  (default_penalty_j if critical_j else 0)
```

4.4 Canonical optimisation proof

```
required = $25,000 − (−$49,700) = $74,700          crunch day 14
Decision variables: 4 AR discounts (x_i), 3 AP deferrals (y_j), 2 financing draws (z_f)
Solver status: Optimal
```

Plan	Composition	Cash raised	Total cost	Day-14 P10 after
Optimal	Zara 1.5% incentive ($645) + Midland Hardware 21-day deferral ($650 = $150 fee + $500 goodwill) + $2,345 Meridian draw ($73.88)	$74,700	**$1,368.88**	**$25,000** ✅
Naive borrowing	$74,700 from QuickFloat @ 46% APR (94.7% effective)	$74,700	$5,812.27	$25,000 ✅
Do nothing	Payroll missed ($1,360 late fee + $18,000 penalty) + Vulcan $6,700 unpaid ($603 + $12,000)	$0	$31,963.00	−$49,700 ❌

```
objective (solver)     = $1,368.88
  cash                 =   $868.88   ($645 discount + $150 deferral fee + $73.88 fee and interest)
  modelled risk        =   $500.00   (0.25 relationship risk × $2,000 price; not money that leaves
the account)
savings_vs_naive       = $5,812.27 − $1,368.88 = $4,443.39   (same objective on both sides)
relationship risk used = 0.25 of 0.40 budget
implied APR on the plan = ($1,368.88 / $74,700) · (365/30) = 22.3%
```

The do-nothing row is **not** a priced alternative and is not quoted as a saving anywhere in the product: its penalty sizes ($18,000, $12,000) are scenario assumptions, and it misses statutory payroll. The UI states the plan cost as cash plus modelled risk, never as a single "cost to fix".

The structural insight the program encodes: **borrowing is the most expensive correct answer.** Working-capital timing levers cost 0.9–2.2% of the cash they free; emergency capital costs 94.7% annualised. The solver reaches for debt only for the $2,345 residual the free levers cannot cover — and it reaches for the *cheapest eligible* facility (Meridian at 38.3%), not the fastest.

4.5 MILP versus a greedy heuristic — measured

The obvious question is whether an integer program is needed at all: with one covering constraint and a per-dollar cost on each lever, sorting levers by cost per dollar raised and filling the gap might be enough. That was measured rather than argued.

`scripts/benchmark-milp-vs-greedy.ts` (`npm run benchmark` , Python engine running) runs every preset × stress-dial combination — 3 × 4 × 3 × 3 × 2 = **216** — through both the HiGHS MILP and the greedy TypeScript mirror and compares plan cost and feasibility. Result, written to `src/lib/benchmark-results.json` and quoted in the optimizer tab:

	count
feasible, identical cost	**144**
feasible, MILP cheaper	**0**
feasible, greedy cheaper	0
infeasible in both	72
feasibility disagreement	0

On every scenario this project ships, the greedy heuristic reaches the same optimum. The MILP is kept for what it does provide — a certificate that the plan is optimal, and a single place where hard constraints are enforced (payroll is never a lever, the relationship-risk budget, the trade-credit insurance cap, integral levers) — not because it finds cheaper plans here. A scenario where indivisible levers or the risk budget make greedy strictly worse would be needed before claiming more, and none is shipped.

---

5. Deterministic TypeScript mirror

`src/lib/quant-client.ts` reproduces both stages in pure TypeScript with no dependencies beyond Node builtins, so the application, its tests and a Vercel preview all function with the Python process down.

Component	Python	TypeScript mirror
PRNG	`np.random.default_rng(42)` (PCG64)	mulberry32 seeded 42
Normals	`scipy.stats.truncnorm`	Box–Muller, clamped at 0
Percentile	`np.percentile` (linear)	same linear interpolation
Optimiser	MILP, HiGHS	greedy cost-per-dollar surrogate

The greedy surrogate takes a binary lever only when its cost beats financing the slice of the gap it covers, which reproduces the MILP optimum on the canonical dataset. It is **not** claimed to be optimal in general, and a test asserts the mirror never reports a cheaper plan than the true MILP. Python is authoritative whenever reachable; the UI badge states which plane served the numbers.

---

6. Scenario presets and the live stress controller

Vantora is single-tenant. There is one plant — **Smart Jeans Manufacturing Ltd.**, Technical Denim & Apparel Plant #4 — and the header carries a stress simulator over it rather than a switcher between unrelated companies. Three one-click presets exist so the same code path has to produce "act now", "do nothing" and "this is not a working-capital problem" without being told which it is. `GET /api/state?profile=…` switches the workspace; it appends nothing to the audit chain.

Preset	Season	Band	Opening cash	Crunch	Solver	Outcome
`SOLVABLE_DAY14_CRUNCH`	Peak order run	**RED**	$52,000	day 14, $74,700	`Optimal`	$1,368.88 three-lever plan
`HEALTHY_RUNWAY`	Balanced season	**GREEN**	$88,000	none	`Optimal`	$0 idle plan; min P10 $77,900 (3.1× buffer)
`INSOLVENT_STRUCTURAL_GAP`	Catastrophic disruption	**BLACK**	$28,000	day 10, $147,000	`INFEASIBLE`	escalate to workout / equity cure

The legacy ids `SME_APEX`, `SME_PRECISION` and `SME_MIDLAND` are aliased to the three presets, so bookmarked links and existing test callers keep resolving.

6.1 The four live stress dials

Each dial perturbs exactly one input and re-runs both the 1,000-trajectory forecast and the MILP:

Dial	Query param	Positions	What it moves
Buyer delay shift	`delayShift`	0 / 7 / 14 / 21	`expected_delay_days_mean` on every AR invoice
Plant shift scale	`workforce`	100 / 125 / 150	the `is_payroll` run, at $680 a head — $68k / $85k / $102k
Systemic correlation ρ	`rho`	0.0 / 0.3 / 0.6	the forecaster's Gaussian copula (§2.2)
Supplier goodwill memory	`priorExtensions`	0 / 1	`past_extension_count` on every flexible payable

At rest, `applyStress` returns the preset object itself, not a structurally equal copy. That is asserted by identity (`toBe`, not `toEqual`), so nothing in the stress path can perturb the dataset every canonical figure is pinned to by re-deriving a field it did not mean to change.

Off-dial values are rejected, not clamped. `parseStressControls` admits only the enumerated positions: `?workforce=1e9` renders the baseline rather than a payroll run nobody designed. Clamping would have silently produced a scenario with no test behind it.

Measured effect of each dial, through the live HiGHS engine:

Controls	Crunch	Gap	Optimal cost	Why
baseline	day 14	$74,700	**$1,368.88**	canonical
`workforce=150`	day 14	$108,700	$2,262.82	+$34,000 of payroll lands on the same day
`priorExtensions=1`	day 14	$74,700	$1,486.79	goodwill spent, so the deferral loses to borrowing on price
`delayShift=14`	day 10	$16,700	$519.42	the book slips, so the breach arrives earlier and shallower
`rho=0.6`	day 14	$74,700	$1,368.88	the crunch-day P10 is structurally invariant (§2.4); the *spread* widens

6.2 Why BLACK matters

The disruption preset's gap is $147,000 against $73,000 of total eligible facility headroom. Drawing every facility to its ceiling still leaves the day-10 obligation unfunded, so the optimiser returns `feasible: false` with no actions rather than a best-effort partial draw. A partial draw would be the recommendation that sells a dying SME a 59% APR advance on the way down; the honest output is that this has stopped being a liquidity problem.

Both engines agree on all three bands — the HiGHS MILP returns `INFEASIBLE` from the `restore_p10_to_safe_buffer` constraint, and the TypeScript mirror reaches the same verdict by exhausting every lever and every facility before reporting a residual gap.

6.3 Statutory payroll breach

`ApPayable.is_payroll` is a declared field, never inferred from the supplier name. When a payroll run goes unpaid on its due date, the do-nothing baseline carries `breaches_statutory_payroll: true` and `feasible: false`. Missing wages is a statutory and operational failure, not a late fee with a price — pricing it at `0.02 × $68,000 + $18,000` and presenting it beside two purchasable alternatives invites the owner to treat it as a third option.

`payroll_lock_day` carries the second half of that constraint. The bank direct-deposit file for a day-14 pay date locks at 2:00 PM on **day 11**, and after it the run cannot be re-cut. The due date is therefore not the deadline, and the verdict bar reads the declared lock day in preference to the engine's generic 72-hour rule.

6.4 Supply-chain corner cases

Four fields change what the solver may do; the rest are declarative provenance the UI renders.

Field	Read by	Effect
`match_status`	seed + `pledgeable_ar`	`QTY_VARIANCE` / `PRICE_VARIANCE` removes the invoice from the factoring borrowing base, and the seed withholds its early-pay eligibility. $52,000 of the $198,000 ledger dilutes out
`past_extension_count` + `extension_surcharge_pct`	`deferral_fee_pct()`	Goodwill is a stock, not a rate. The first schedule change carries no cash surcharge; the next adds a step
`insurance_revocation_day_cap`	`deferral_days()`	Trade-credit cover (Atradius/Coface) is withdrawn past the cap, converting the supplier to cash-in-advance and triggering bureau reporting. The window is clamped before the solver sees it, so that outcome is never a priced trade-off

Field	Read by	Effect
`is_flexible: false` on `MONOPOLY_STRICT`	the MILP lever loop	A single-source supplier whose dies stop the line is not expensive to defer, it is undeferrable. Modelled as the absence of a lever rather than a large fee the optimiser could decide to pay

The goodwill step is the one worth watching in the demo. Setting `priorExtensions=1` adds $450 to Midland's deferral — and the optimiser responds by **dropping the deferral entirely** and taking Urban Retail's discount plus a smaller draw, for $1,486.79. It does not quietly pay the surcharge. That is the MILP re-solving, not a branch anyone wrote.

---

7. The cost of procrastination

The gap never moves. What expires is the *means* of closing it. Each decision horizon re-runs the optimiser against the same forecast with only the levers an owner who waited that long still holds:

```
days_remaining(w) = crunch_day − w

AR discount i open   ⟺  settles_in_days_i + AR_OFFER_LEAD_DAYS   ≤ days_remaining(w)      lead = 2
AP deferral  j open  ⟺  due_day_j − w                            ≥ AP_DEFERRAL_NOTICE_DAYS  notice = 6
Facility     f open  ⟺  funding_daysf                            <  days_remaining(w)

cost(w) = optimise( state with only the open levers, same forecast )
```

The two lead-time constants encode what a counterparty needs rather than what the schedule permits: an early-payment offer has to be read, approved and queued before it settles, and a supplier will not agree to move a bill that falls due this week.

Day waited	Days left	Cost	What closed
0 (today)	14	**$1,368.88**	— every lever open
4	10	**$1,368.88**	Metro Power & Gas deferral
7	7	**$1,486.79**	**Midland Hardware deferral** + Indigo Denim Mill X; Granite (7d funding)
11	3	**$2,353.56**	all three AR discounts — Zara, Urban Retail, Northside
13	1	**INFEASIBLE**	Meridian and QuickFloat (2d funding, 1 day left)

Day 11 is the payroll file lock (§6.3). Past it every working-capital lever has closed and the whole $74,700 has to be borrowed on the confidential line: `0.02 × 74,700 + 74,700 × 0.14 × (30/365) = $2,353.56`, a **1.7×** cost increase for eleven days of delay — and that is the *cheap* outcome, available only because Meridian is already underwritten and funds in two days. By day 13 nothing can fund in time at any price.

The curve is computed, not authored. Each row is a real solver call, which is why Day 7 is $1,486.79 rather than the $1,664.09 a hand-worked "Meridian covers the $30k" narrative would predict — Urban Retail's discount is still open at seven days out, and the optimiser finds it.

---

8. Plan resilience — refusal, echo and the live probe

A plan that is optimal on paper has two questions left to survive, and both are computed from the same forecast the optimiser used, so neither can move a canonical figure (`src/lib/recovery-scenarios.ts`).

8.1 Counterparty refusal — Plan B

Every lever in the plan except the bridge draw is an **offer**. Zara can decline the 1.5%; Midland can refuse the 21-day move. `counterpartyFallbacks()` withdraws each counterparty-dependent lever and re-solves against the already-computed forecast, so the premium is a mirror-to-mirror delta rather than two different solvers compared:

Branch	Withdrawn	Re-solved cost	Premium
`buyer_declines`	INV-2002 early-pay discount	**$1,837.50**	+$468.62
`supplier_refuses`	AP-RAW-STEEL deferral	**$1,486.79**	+$117.91
`both_decline`	both	**$2,109.42**	+$740.54

All three stay feasible. The substitution is the interesting part: **Urban Retail's 2.5%/4-day discount loses to borrowing on price in Plan A and becomes the cheapest cash left in Plan B.** Even losing both key counterparties, the gap closes for $2,109.42 — an order of magnitude below the $31,963.00 cost of inaction.

Branches are derived from the levers the plan actually selected, not from hard-coded counterparty ids, so `HEALTHY_RUNWAY` (no plan to defend) and `INSOLVENT_STRUCTURAL_GAP` (no feasible plan) both correctly return `[]` rather than inventing refusals.

8.2 Day 15–45 echo solvency check

Pulling a receivable forward and pushing a payable back is borrowing from the near future. `postCrunchEchoCheck()` walks the window deterministically from the post-recovery balance and books every obligation on the existing ledger, including the two the plan itself created:

```
opening            $25,000   restored P10 at the crunch day
d22   -$21,000     quarterly VAT remittance        → $4,000     pre-existing
d26   +$31,000     INV-2003 Urban Retail           → $35,000
d26   +$52,000     INV-2001 Kenneth Cole           → $87,000    CONTINGENT (GR/IR blocked)
d31   -$30,000     AP-RAW-STEEL, deferred from d10 → $57,000    plan-created
d44    -$2,418.88  Meridian principal + interest   → $54,581.12 plan-created
```

Two deliberate modelling choices:

1. **No next-cycle revenue is assumed.** A solvency proof that leans on sales nobody has made yet proves nothing, so every inflow in the walk is a named invoice already on the book. A test asserts exactly that.
2. **Contingent inflows are tracked separately, not discounted into the balance.** A receivable sitting in a buyer's match-exception queue is not 80% of itself; it either reconciles or it does not. The walk is therefore reported **both ways**, and the UI headlines the conservative one.

The honest conclusion is narrower than "the plan is safe":

- **Coverage limit first.** The ledger is a 30-day book with no recurring obligations. Next month's payroll, rent and utilities would fall inside days 31–45 if they recur monthly, and they are not in the data — so this check **cannot** say days 31–45 are solvent, and the UI does not say so. It says what it can see.
- **Every recorded obligation in the window is funded.** Cash stays positive on both reads, including the two obligations the plan itself creates (Midland on day 31, the Meridian repayment on day 44).

- **The buffer does dip.** Day 22 falls to $4,000 on the VAT remittance — which is on the book in the do-nothing world too, so it is a pre-existing trough, not a cost of the recovery.
- **Both plan-created days clear the buffer only if the blocked match reconciles.** Withholding Kenneth Cole's $52,000, day 31 is $5,000 and day 44 is $2,581.12.

That dependency is the finding. It ties the echo window directly to the clerk-signed 3-way reconciliation draft, which is why that Tier 1 action is offered even though the solver cannot price it (§6.4).

8.3 Live security probe

`POST /api/action` with `action: "custom_security_probe"` runs the injection screen and the numerical attestation over caller-supplied text and returns both verdicts. It calls the two pure screens directly rather than `enforcePermissionGate`, so probing is read-only by construction: no amount of poking grows the hash chain, and the submitted text is never logged or persisted. Input is capped at 4,000 characters and non-string input is rejected at the boundary. Deliverable 05 §6.5 documents the control; the dashboard surfaces it as the judge-operated playground on Tab 4.

---

9. Identity, approval and the sandbox position

The quantitative plane produces a plan; this layer decides who may act on it, binds an approval to exactly what was approved, and keeps the plan honest about what has already happened. Code: `src/lib/server/`.

9.1 Identity and roles

Better Auth (email + password, scrypt hashes, 8-hour sessions with hourly sliding refresh) stores users and sessions in a local SQLite file (`data/vantora.db`, `node:sqlite`). Roles are **not** session claims: every request re-reads the `membership(user_id, business_id, role)` row, so a demotion or revocation takes effect immediately (`npm run users -- set-role|revoke` also deletes the user's sessions). There is no sign-up endpoint: a database hook refuses user creation outside the owner-run CLI, and the HTTP sign-up route answers 403.

Capability	OWNER	ANALYST	VIEWER	Copilot (service identity)
read dashboard, ledger, audit chain	✓	✓	✓	✓
analyse (copilot questions, red-team drill)	✓	✓	–	✓ (within the human's role)
draft (Tier 1)	✓	✓	–	✓ (within the human's role)
approve / execute (Tier 2)	✓	–	–	never approves; executes only an owner delegation
record simulated counterparty responses	✓	–	–	–

9.2 The canonical plan and its version

`loadPlanContext(business, scenario)` derives everything an action needs on the server, from four inputs, and hashes them into the **plan version**:

```
plan_version = sha256( dataset_version, scenario_key, sandbox position, re-solved actions in cents
)[:16]
dataset_version = sha256( every scenario preset )[:12]          # a seed edit invalidates all
approvals
scenario_key    = dataset | preset | delayShift | workforce | rho | priorExtensions
```

Each action becomes a canonical payload in **integer cents** (`face_cents` , `discount_cents` , `deferred_cents` , `fee_cents` , `principal_cents` , …) with an action key that identifies the real-world action, independent of plan version. The browser never supplies an amount the server trusts; it names an action key.

9.3 Approval and execution

1. **Approve (OWNER).** Re-derive the plan for the scenario the browser shows; look the action up by key; re-validate it against the ledger and the financing screen (facility exists and passed screening, principal within its limit, payable flexible and in the plan, invoice clean and discount-eligible, plan feasible); require the owner to type the figure that matters for that action — `discount_cents` , `deferred_cents` , `principal_cents` , or the plan's `cash_cost_cents` — and compare it as an integer. Insert an approval row bound to approver, business, scenario, plan version and the payload digest (5-minute expiry, single use, optional delegation to the copilot), and hand the browser an HMAC token carrying the row id.
2. **Execute (OWNER, or the copilot on a delegation).** Re-derive and re-validate again, verify the token, then — synchronously, inside one `BEGIN IMMEDIATE` transaction — check the row (same user or delegated, unexpired, unconsumed, unrevoked, same scenario, same plan version, same digest), burn it, and insert one `action_execution` row per action. The action key is the primary key, so a replay, a concurrent duplicate, or the same action reached through the full-plan and the single-action paths cannot run twice.
3. **Refusals are recorded.** A refused approve/execute by an authenticated owner is appended to the audit chain with its reason.

9.4 Lifecycle and position-aware re-planning

Kind	States
Early-payment offer	PROPOSED → APPROVED → SUBMITTED → AWAITING_COUNTERPARTY → ACCEPTED → SETTLED, or DECLINED / FAILED
Supplier deferral	PROPOSED → APPROVED → SUBMITTED → AWAITING_COUNTERPARTY → ACCEPTED (due date moved), or DECLINED / FAILED
Facility drawdown	PROPOSED → APPROVED → SUBMITTED → AWAITING_COUNTERPARTY → ACCEPTED → SETTLED, or DECLINED / FAILED

Before re-solving, the sandbox position is applied to the ledger. In-flight actions become **forecast** effects — an offer becomes a deterministic inflow on its settlement day, a deferral moves its due date, a submitted draw becomes an `expected_funding` inflow that is never collateral and never discountable — and only `SETTLED` actions change `current_cash` (recorded separately as cash movements, at most one per action: `cash_movement_once` is a unique index on the action key). The execution queue shows the resulting **sandbox account**: cash (opening and after settlement), receivables outstanding, the discount given up, the facility balance owed with its repayment day, bills whose due date moved, and pending inflows that are not cash yet. A declined or failed action removes its lever. The plan is then re-forecast and re-solved, which has two visible consequences in the canonical scenario:

- After the full plan is submitted, day 14 is projected back at the $25,000 buffer **if every counterparty accepts**, and the re-solve surfaces the **next** shortfall: day 22, $21,000 below the buffer (the VAT remittance the echo check had flagged), funded by the Heritage SBA facility that was too slow for day 14.

- If Zara then declines, day 14 reopens at $42,355 below the buffer, every outstanding approval in the scenario is revoked, and the revised plan substitutes Urban Retail's 2.5%, Northside's 3% and a second $490 Meridian draw ($1,150.44 cash).

9.5 AI copilot (Google Gemini)

`agent/gemini.ts` runs a manual, bounded function-calling loop against `GEMINI_MODEL` (default `gemini-3.1-flash-lite`): the model's content is appended verbatim to the history (keeping thought signatures attached to their calls), each call is answered with a `functionResponse` carrying its `id`, arguments are validated against the tool's JSON schema, unknown tools and unexpected arguments are refused, at most six calls per turn and five model turns are allowed, a 20-second timeout applies, and provider calls are never retried. Every tool call is dispatched by `server/agent-tools.ts` under the signed-in human's authority, capped at the copilot's own service capabilities; Tier 2 tools go through the approval service as an agent executor and succeed only on an owner delegation for that exact action. The final text must pass numerical attestation or it is withheld. The response always states its source: `gemini` with the model id, or `deterministic` with a reason (`not_configured`, `provider_error`, `timeout`, `integrity_failure`, `tool_loop_limit`, …). Live verification against the Gemini API is pending a configured key; the loop and the policy boundary are covered by mocked tests.

---

10. Reproducing every figure

```
npm install
cp .env.example .env    # set BETTER_AUTH_SECRET and HITL_SIGNING_SECRET
npm run users -- bootstrap-owner --email owner@example.com --name "Plant Owner"
npm run dev:all         # Next.js :3000 + FastAPI :8000

npx vitest run                                                    # 172 tests
quant_engine/.venv/Scripts/python.exe -m pytest quant_engine/tests -q # 34 tests
npm run lint && npx tsc --noEmit && npm run build                 # clean, zero warnings
npm run benchmark                                                 # §4.5
```

206 tests total (172 Vitest, 34 pytest). With the Python engine stopped, two of the Vitest cases are live-bridge comparisons that skip themselves and everything else still passes against the TypeScript mirror.